Guía de exposición — Wilfero Delgado

Curso: Desarrollo Web · **Proyecto:** Vivent · **Ciclo:** octubre 2026

Documento de apoyo individual. No sustituye al `DOCUMENTO_PROYECTO.md`; lo resume desde tu rol.

---

0. Tu rol en una tabla

Elemento	Lo que te corresponde
Módulos (2)	Compras · Ventas POS
Proceso integrado (1)	P2 — Venta confirmada → descuento FIFO → kardex → cierre de caja
Reportes (3)	Ventas por período · Compras por proveedor · Cierre de caja
API externa (1)	**Open Food Facts** (catálogo externo por código de barras)
Transversales	Dashboard · Docker y publicación
Carpetas de Postman	`01 · Autenticación`, `04 · Terceros`, `06 · Compras`, `07 · Ventas`, `09 · Cierres de caja`, `10 · Reportes`, `11 · Integraciones externas` (Open Food Facts)

> Eres el único que toca la **caja**: el cierre es el que cierra el ciclo venta → cobro → arqueo. Coordina con Fernando que el traslado ya esté recibido antes de tu cierre, para que los números cuadren.

---

1. Guion de 12 minutos

Min	Qué muestras	Qué dices
0–1	Dashboard (`/dashboard`)	"Los indicadores salen de la base, no están escritos: ventas del día, stock bajo, movimientos recientes."
1–3	Compras → nueva compra → elegir proveedor, producto y presentación → guardar borrador → confirmar	"La compra nace en `DRAFT`; al confirmar se crea la capa FIFO con su costo y sube el inventario. Una compra confirmada ya no se edita: se anula con motivo."
3–6	POS (`/sales/new`) con usuario `seller` → escanear `740000000001` (o pegar el código) → cambiar cantidad → cobrar	"El POS resuelve la presentación por código de barras, valida stock disponible y al confirmar consume FIFO. El descuento a nivel venta exige `sales.apply_discount`."
6–8	Ventas → abrir la venta → **Anular** con motivo y disposición «regresar a inventario»	"La anulación devuelve la mercadería, deja el movimiento `SALE_CANCEL_IN` en kardex y queda en auditoría con el motivo."
8–10	Cierres → **Preview del día** → crear cierre → aprobar	"El preview calcula el esperado por método de pago antes de cerrar; el cierre consolida ventas, anulaciones y descuentos del día."
10–11	Reportes: Ventas por período con rango de fechas → **Exportar CSV** → **Imprimir**	"Filtra en el servidor por fecha, sucursal, cliente, estado y método de pago."
11–12	Postman `11` → Open Food Facts con `3017620422003` y con `0000000000000`	"Si el código no existe en el catálogo local, el POS consulta Open Food Facts y ofrece registrar el producto; si tampoco existe allá, devuelve `found=false` sin romper la venta."

---

2. Postman: qué ejecutar y en qué orden

#	Request exacto	Resultado esperado	Qué demuestras
1	`01 · Autenticación → Login (guarda tokens)`	`200`	Sesión
2	`01 · Autenticación → Refresh token (renueva y rota)`	`200` con par nuevo	Renovación (tu parte del JWT)
3	`01 · Autenticación → Logout (revoca refresh)`	`200`	Cierre de sesión real
4	`04 · Terceros → Proveedores → Crear proveedor`	`200`	CRUD de terceros
5	`06 · Compras → Crear compra (DRAFT)`	`200` con `purchaseId`	Estado inicial
6	`06 · Compras → Confirmar compra`	`200` estado `CONFIRMED`	FIFO + kardex
7	`06 · Compras → Anular compra`	`200` estado `VOIDED`	Anulación con motivo
8	`07 · Ventas → Confirmar venta`	`200`	Consumo FIFO
9	`07 · Ventas → Anular venta`	`200`	Devolución a inventario
10	`09 · Cierres de caja → Preview del día`	`200` con totales	Arqueo previo
11	`09 · Cierres de caja → Crear cierre`	`200` estado `CLOSED`	Cierre del día
12	`10 · Reportes → Resumen de ventas`	`200`	Reporte de ventas
13	`11 · Integraciones externas → Open Food Facts: código de barras`	`found: true`	API externa
14	`11 · Integraciones externas → Open Food Facts: inexistente (error)`	`found: false`	Manejo de error

> El request `09 · Crear cierre` calcula una fecha pasada libre en un pre-request, porque la regla es **un cierre por usuario + sucursal + día**.

---

3. Código: dónde apuntar

Tema	Archivo	Punto exacto
Crear compra	`backend/.../purchases/service/PurchaseService.java`	`createPurchase()` → `applyPurchaseFields()` + `recalculateTotals()`
Confirmar compra (FIFO)	`backend/.../purchases/service/PurchaseService.java`	`confirmPurchase()` → llama a `inventoryService.receiveStock(...)`
Anular compra	`backend/.../purchases/service/PurchaseService.java`	`voidPurchase()` exige motivo y revierte la capa
Confirmar venta	`backend/.../sales/service/SaleService.java`	`confirmSale()` → `inventoryService.issueStock(...)`; guarda `fifoCostTotal`

Tema	Archivo	Punto exacto
Anular venta	`backend/.../sales/service/SaleService.java`	`cancelSale()` valida la disposición (`RETURN_TO_STOCK` , `LOSS` , `DAMAGE`)
Descuentos	`backend/.../sales/service/SaleService.java`	`ensureDiscountPermission()`
POS (frontend)	`frontend/src/features/sales/components/sale-pos-screen.tsx`	Escaneo, líneas, totales y cobro
Escáner	`frontend/src/features/sales/components/mobile-barcode-scanner.tsx`	Entrada por cámara/código
Validación del formulario	`frontend/src/features/sales/schemas/sale-schema.ts`	Zod antes de enviar
Cierre de caja	`backend/.../closures/service/CashClosureService.java`	`previewClosure()` , `createClosure()` , `approveClosure()` , `reopenClosure()` , `closeClosure()`
Reportes que expones	`backend/.../reports/service/ReportService.java`	`salesSummary()` , `purchasesSummary()` , `cashClosuresSummary()`
Open Food Facts	`backend/.../barcodes/service/BarcodeLookupService.java`	`lookup()` con caché 10 min
Renovación de sesión (frontend)	`frontend/src/lib/api/http.ts`	Renovación proactiva + reintento ante `401` (single-flight)
Docker	`docker/compose.local.yml`	db 5433 · backend 8080 · frontend 3000
Publicación	`docs/PUBLICACION_CLOUD.md`	Neon + Render + Vercel

---

4. Tus tres reportes

Reporte	Ruta	Endpoint que consume	Filtros que debes mover
Ventas por período	`/reports/sales`	`GET /api/v1/reports/sales-summary`	Fechas, sucursal, cliente, estado, método de pago
Compras por proveedor	`/reports/purchases`	`GET /api/v1/reports/purchases-summary`	Fechas, proveedor, estado
Cierre de caja	`/reports/closures`	`GET /api/v1/reports/cash-closures-summary`	Fechas, sucursal, estado

Guion de 60 segundos: pon un rango de fechas que **incluya** las ventas de hoy → muestra el total → **Exportar CSV** ("solo lo filtrado") → **Imprimir** ("vista de impresión sin menú").

> Cuidado con el formato de fecha: los reportes de ventas, productos, traslados y ajustes reciben fecha-hora ISO; los de compras y cierres reciben fecha simple. En la pantalla no se nota porque `reports-api.ts` lo resuelve, pero si lo pruebas a mano en Postman, importa.

---

5. Tu API externa: Open Food Facts

Dato	Valor
Endpoint	`GET /api/v1/products/barcode-lookup/3017620422003`
Código	`backend/.../barcodes/controller/BarcodeLookupController.java` + `service/BarcodeLookupService.java`
Respuesta de éxito	`{ barcode, source: "OPEN_FOOD_FACTS", found: true, name: "Nutella", brand, quantityText, imageUrl }`
Respuesta de error	`{ found: false, message }` — nunca lanza excepción
Protecciones	Caché 10 min · timeout 4 s · requiere `products.view`

Qué decir: "Open Food Facts es una base abierta de productos. La uso como **fallback**: primero busco en el catálogo propio; si el código no está, consulto la API externa y el POS ofrece crear el producto con nombre, marca y presentación ya llenos. Si la API no responde, el vendedor sigue pudiendo buscar el producto a mano."

Demo: código `3017620422003` (Nutella) → `found: true`; código `0000000000000` → `found: false`.

---

6. Rúbrica: cómo la cumples tú

Criterio de la rúbrica	Cómo se cumple	Evidencia que muestras
2 módulos funcionales	Compras (borrador → confirmar → anular) y Ventas POS (escaneo, descuento, cobro, anulación)	Recorrido en vivo + `06` / `07` de Postman
1 proceso integrado	P2: venta confirmada → FIFO → kardex → cierre de caja	Venta confirmada + cierre del día
3 reportes con filtros	Ventas, Compras, Cierre de caja	Un filtro aplicado en cada uno
Impresión / exportación	Botones de `report-actions.tsx`	CSV descargado + vista previa
1 API externa	Open Food Facts con éxito y error	Requests de la carpeta `11`
≥5 registros por catálogo	Seed académico + proveedores/clientes	Listados con el total
JWT access + refresh	Tu parte: **renovación, rotación y logout** (`http.ts` + `RefreshTokenService`)	Login → Refresh → Logout en Postman
CRUD en Postman	Compras, ventas, cierres y terceros	Run de las carpetas
Docker y despliegue	`compose.local.yml` + guía de nube	`docker ps` + `PUBLICACION_CLOUD.md`
Commits propios	Pendiente: debes subir tu propia contribución	`git log --author="Wilfero"`

---

7. Preguntas probables

1. **¿Por qué la compra confirmada no se edita?** Porque ya generó una capa FIFO y kardex; editarla reescribiría la historia. Se anula con motivo y se crea otra.
2. **¿Cómo se evita vender sin stock?** `issueStock()` valida `onHand − reserved ≥ cantidad` y lanza `INSUFFICIENT_STOCK` sin escribir nada.
3. **¿El descuento lo puede hacer cualquier vendedor?** El de línea sí; el descuento sobre el total exige `sales.apply_discount` (solo admin), validado en `ensureDiscountPermission()`.
4. **¿Qué pasa con la mercadería de una venta anulada?** Se elige la disposición: regresa a inventario (`SALE_CANCEL_IN`), se manda a pérdida o a dañado, cada una con su movimiento de kardex.
5. **¿El cierre recalcula o copia?** Recalcula: `previewClosure()` consulta ventas y pagos del día; el cierre guarda el snapshot con el esperado y la diferencia contra lo contado.
6. **¿Puede haber dos cierres del mismo día?** No para el mismo usuario y sucursal: índice único y validación en el servicio.
7. **¿Qué pasa si el refresh token se roba?** Es opaco, se guarda hasheado (SHA-256) y se rota en cada uso; además se revoca al cambiar contraseña y en logout.
8. **¿La venta emite factura?** No: el alcance del curso no incluye facturación legal; se registra la venta interna con método de pago y cierre.

---

8. Checklist 10 minutos antes

- [ ] `docker ps` con los tres contenedores arriba.
- [ ] `python database/bootstrap_demo.py` corrido (necesitas ventas y un cierre del día).
- [ ] Usuario `seller` probado (contraseña `ChangeMe123!`) para el POS.
- [ ] Un código de barras a mano: `740000000001` (demo) y `3017620422003` (Nutella).
- [ ] Postman con login ejecutado (los requests heredan `accessToken`).
- [ ] `documento-proyecto.pdf` abierto en §5.3–5.4 y §7.4 (flujo de venta).
- [ ] Panel del dashboard cargado (evita esperar la primera consulta en vivo).